

Advanced Signal Processing

Exam questions

Abstract

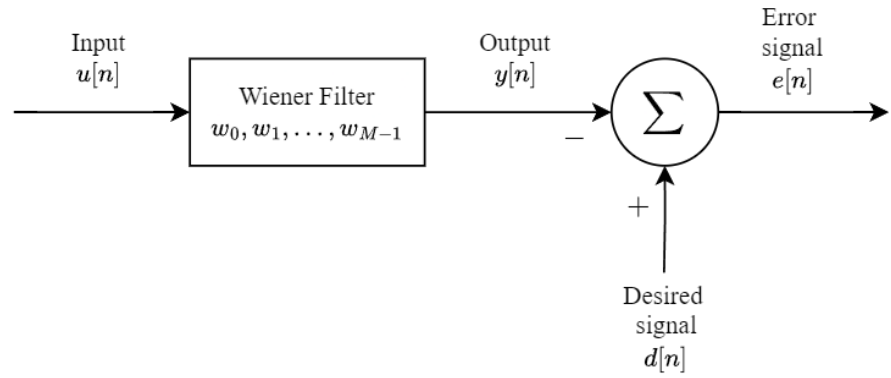
The oral exam lasts about 15 minutes per student, with 5 additional minutes for grading. Students randomly draw two topics from two different course areas (A: 1-4, B: 5-8, or C: 9-12) at the start of the exam and choose the order of discussion. They present their understanding of each topic, supported by a sheet of notes and using the blackboard for visuals, while examiners ask questions. The exam is graded on the 7-step scale and conducted in English or Danish by the course lecturers.

Contents

1. Wiener Filtering	2
2. Least-Mean-Squares Adaptive Filtering	3
3. RLS Adaptive Filtering	4
4. Multidimensional Fourier Transform.....	5
5. Signal Spaces	6
6. Detection theory.....	7
7. Minimum Mean Squared Error (MMSE) and Linear MMSE estimation.....	8
8. Kalman Filtering (scalar and vector versions)	9
9. One-step linear predictor.....	10
10. The Levinson-Durbin algorithm	11
11. AR-filter design by modelling.....	12
12. The Lattice FIR filter	13

1. Wiener Filtering

Wiener filtering is a powerful linear technique used to minimize the mean squared error (MSE) between a desired signal $d(n)$ and the filter output $y(n)$. It is ideal for stationary signals, where signal statistics remain constant over time. The Wiener filter operates by finding an optimal set of weights w that minimizes the error $e(n)$ between $d(n)$ and $y(n)$.



1. The filter output is computed as:

$$y(n) = \sum_{k=0}^{\infty} w_k^* u(n-k)$$

$$\text{ex. 1 FIR filter with } M \text{ weights } \sum_{i=0}^{M-1} w_{oi} r(i-k) = p(-k)$$

where $u(n)$ represents the input signal and w_k are the filter weights.

The error signal is defined as:

$$e(n) = d(n) - y(n)$$

2. The optimization goal is to minimize the MSE:

$$J = \mathbb{E}[|e(n)|^2]$$

Steepest descent

- Assumes knowledge of signal statistics (R, p).
- Optimal convergence to Wiener Solution as $n \rightarrow \infty$
- On Wiener-filtering (update rule):

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu[\mathbf{p} - \mathbf{R}\mathbf{w}(n)]$$

3. To achieve this, the Wiener-Hopf equations are used:

$$\mathbf{R}\mathbf{w}_o = \mathbf{p}, \quad \mathbf{w}_o = \mathbf{R}^{-1}\mathbf{p} \quad (\text{matrix form})$$

The **principle of orthogonality** ensures that, at optimality, the error $e(n)$ is orthogonal to all components of the input $u(n)$. This property guarantees that the filter minimizes the MSE.

where $\mathbf{R} = \mathbb{E}[\mathbf{u}\mathbf{u}^H]$ is the autocorrelation matrix of the input signal

$\mathbf{p} = \mathbb{E}[\mathbf{u}d^*]$ is the cross-correlation vector between the input $u(n)$ and the desired signal $d(n)$

$\mathbf{w}_o = (w_{o0}, w_{o1}, \dots, w_{o,m-1})^T$ (Optimal weights)

While the Wiener filter is effective, its implementation requires knowledge of $\mathbf{R}$ and $\mathbf{p}$, which must be estimated in practice. Additionally, computing $\mathbf{R}^{-1}$ can be computationally expensive for high-dimensional systems.

Applications: Wiener filters are widely used in noise cancellation, signal prediction, and system modeling.

Extra:

1. Wiener optimal solution

$$\nabla_k J = \nabla_k \mathbb{E}[e(n)e^*(n)] = -2\mathbb{E}[u(n-k)e^*(n)]$$

$$\nabla_k J = \mathbb{E}[u(n-k)e_o^*(n)] = 0$$

Principle of orthogonality: when the filter operates optimally (i.e., J is minimum), $e_o(n)$ is orthogonal to each input sample $u(n-k)$ contributing to the estimation of the desired response.

2. Wiener-Hopf equations

From the Wiener optimal solution, we can expand by inserting e_o^* :

$$\mathbb{E} \left[u(n-k) \left(d^*(n) - \sum_{i=0}^{\infty} w_{oi} u^*(n-i) \right) \right] = 0$$

$$\sum_{i=0}^{\infty} w_{oi} \mathbb{E}[u(n-k)u^*(n-i)] = \mathbb{E}[u(n-k)d^*(n)]$$

This leads to the Wiener-Hopf equations

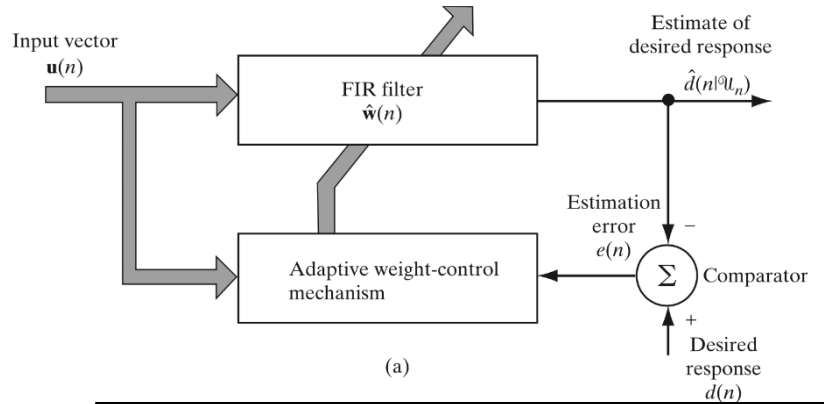
$$\sum_{i=0}^{\infty} w_{oi} r(i-k) = p(-k), \quad \text{ex 1 } \uparrow$$

With: $\mathbb{E}[u(n-k)u^*(n-i)] = r(i-k)$

$$\mathbb{E}[u(n - k)d^*(n)] = p(-k)$$

2. Least-Mean-Squares Adaptive Filtering

LMS adaptive filtering is a method to minimize the mean squared error between a desired signal and the filter output. It adjusts filter coefficients iteratively using stochastic gradient descent (SGD), making it simple and computationally efficient for real-time applications.



1. The goal of LMS filtering is to minimize the MSE cost function:

$$J = \mathbb{E}[|e(n)|^2] = \mathbb{E}[|d(n) - \mathbf{w}^H(n)\mathbf{u}(n)|^2]$$

2. Approximation Using Instantaneous Error:

$d(n)$: Desired signal

$e(n)$: Error between the desired sig. and the filter output.

Instead of using the expectation operator, LMS minimizes the instantaneous squared error:

$$J_s(n) = |e(n)|^2 = e(n)e^*(n)$$

3. Gradient Computation:

The gradient of the cost function $J_s(n)$ with respect to the filter weights $\mathbf{w}(n)$ is:

$$\nabla J_s(n) = \frac{\partial J_s(n)}{\partial \mathbf{w}_k^*(n)} = -2u(n - k)e^*(n)$$

4. Weight Update (Minimization Step):

To minimize $J_s(n)$, the filter weights are updated in the direction of the negative gradient:

$$\hat{\mathbf{w}}_k(n + 1) = \hat{\mathbf{w}}_k(n) - \frac{1}{2}\mu \nabla J_{s,k}(n)$$

Substituting $\nabla J_{s,k}(n)$, the update rule becomes:

$$\hat{\mathbf{w}}_k(n + 1) = \hat{\mathbf{w}}_k(n) + \mu \cdot u(n - k)e^*(n)$$

5. Stability Criterion:

As for steepest descent, convergence/stability of LMS is guaranteed when: $0 < \mu < \frac{2}{\lambda_{max}}$. Where λ_{max} is the largest eigenvalue of the input signal's autocorrelation matrix. In practice, μ has to be sufficiently small for a robust behavior **NOTICE: Instantaneous estimates for $\mathbf{R}$ and $\mathbf{p}$ are included by the feedback and averaged over time.**

Normalized LMS: LMS can suffer from unstable updates when the input signal has high power. NLMS resolves this by normalizing the step size with the signal power i.e. $\|\mathbf{u}(n)\|^2$:

$$\hat{\mathbf{w}}(n + 1) = \hat{\mathbf{w}}(n) + \frac{\mu}{\delta + \|\mathbf{u}(n)\|^2} \cdot \mathbf{u}(n - k)e^*(n)$$

Where the δ is included for numerical stability. NLMS reduces the risk of large updates and ensures faster convergence.

3. RLS Adaptive Filtering

The Recursive Least-Squares (RLS) algorithm minimizes the weighted least squares error between a desired signal $d(n)$ and the filter output $y(n)$. It uses past data with exponentially decreasing weights, enabling fast convergence and adaptability to non-stationary signals.

1. Minimize Cost Function: The RLS algorithm minimizes a least-squares cost function $J_2(\mathbf{w})$, which sums the squared errors over a data segment:

$$J_2(\mathbf{w}) = \sum_{i=i_1}^{i_2} e^2[i] = \sum_{i=i_1}^{i_2} (d[i] - y[i])^2$$

i_1 and i_2 :
segment's
length
 $\mathbf{w}$: filter
weights.

2. Weighted Cost Function: To prioritize recent data, the RLS algorithm uses a weighted cost function with a weighting factor ($\beta[n, i]$):

$$J_\beta(\hat{\mathbf{w}}[n]) = \sum_{i=1}^n \beta[n, i] e^2[i]$$

$$= \sum_{i=1}^n \beta[n, i] (d[i] - \hat{\mathbf{w}}^T[n] u[i])^2$$

λ : forgetting
factor.
Assigning
less weight
to older data
↓

3. Exponential Weighting: The weighting function ($\beta[n, i]$) can take the form of the exponential function:

$$\beta[n, i] = \lambda^{n-i}, \quad 0 < \lambda \leq 1, \quad i = 1, 2, \dots, n$$

4. Regularization Term: To prevent noise from affecting early updates, a regularization term δ is added to the cost function:

$$J_\beta(\hat{\mathbf{w}}[n]) = \sum_{i=1}^n \lambda^{n-i} e^2[i] + \delta \lambda^n |\hat{\mathbf{w}}[n]|^2$$

5. Gradient: The gradient of this cost function with respect to the filter weights is calculated and set to zero, resulting in the following system of equations:

$$\nabla J_\beta(\hat{\mathbf{w}}[n]) = 0$$

$$\sum_{i=1}^n \lambda^{n-i} \mathbf{u}[i] \mathbf{u}^T[i] \hat{\mathbf{w}}[n] + \delta \lambda^n \hat{\mathbf{w}}[n] = \sum_{i=1}^n \lambda^{n-i} \mathbf{u}[i] d[i]$$

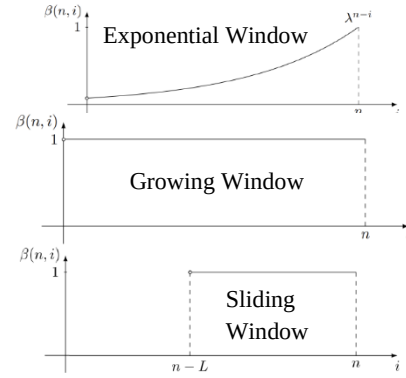
$$\Phi[n] \hat{\mathbf{w}}[n] = \mathbf{z}[n]$$

$\Phi[n]$ is an $M \times M$ time-average auto-correlation matrix of the filter input. $\mathbf{z}[n]$ is an $M \times 1$ time-average cross-correlation vector.

$$\beta(n, i) = \begin{cases} \lambda^{n-i} & 0 < i \leq n \\ 0 & \text{otherwise} \end{cases}$$

$$\beta(n, i) = \begin{cases} 1 & 0 < i \leq n \\ 0 & \text{otherwise} \end{cases}$$

$$\beta(n, i) = \begin{cases} 1 & n-L < i \leq n \\ 0 & \text{otherwise} \end{cases}$$



To solve the weighted normal equations:

$$\Phi[n] \hat{\mathbf{w}}[n] = \mathbf{z}[n]$$

One must solve:

$$\hat{\mathbf{w}}[n] = \Phi^{-1}[n] \mathbf{z}[n]$$

6. Correlation Matrix: Using exponential weight, the recursive computation of the correlation matrix $\Phi[n]$ is:

$$\Phi[n] = \lambda \Phi[n-1] + u[n] u^T[n]$$

This equation states that the present value of the correlation matrix $\Phi[n]$ is computed by the old matrix $\Phi[n-1]$ and the outer product of the newly arrived input sequence. The same approach for recursive computation of the vector $\mathbf{z}[n]$ leads to:

$$\mathbf{z}[n] = \lambda \mathbf{z}[n-1] + u[n] d[n]$$

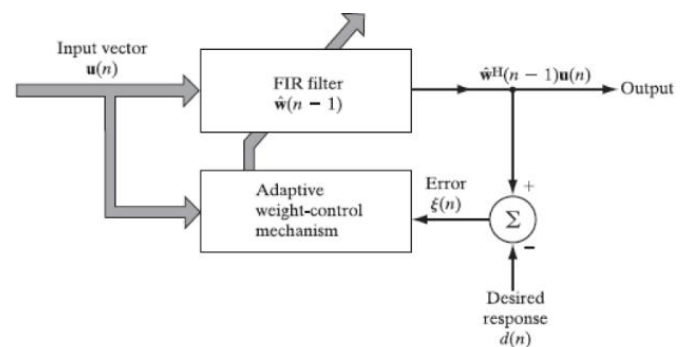
8. Matrix Inversion: After calculating $\Phi[n]$ and $\mathbf{z}[n]$, the filter weights are obtained by inverting the matrix $\Phi[n]$ and solving the system. However, this can be computationally expensive, especially for large filter lengths. To address this, the Matrix Inversion Lemma is used giving us:

$$\Phi^{-1}[n] = \lambda^{-1} \Phi^{-1}[n-1] - \lambda^{-1} k[n] u^T[n] \Phi^{-1}[n-1]$$

9. Weight Update: The filter weight update is given by:

$$\hat{\mathbf{w}}[n] = \Phi^{-1}[n] \mathbf{z}[n] = \hat{\mathbf{w}}[n-1] + \mathbf{k}[n] \xi[n]$$

$\mathbf{k}[n]$: Gain vector, determines the weight adjustment
 $\xi[n]$: Priori estimation error



4. Multidimensional Fourier Transform

The multidimensional Fourier transform generalizes the Fourier transform to signals defined in multiple dimensions. It is a critical tool for analyzing spatial and temporal signals in fields like imaging, radar, telescopes and medical diagnostics.

1. Fourier Series

Assume a function f defined over a finite interval T :

$$f(t) = \sum_{k=-\infty}^{\infty} a_k e^{i2\pi k \frac{t}{T}}, \quad a_k = \frac{1}{T} \int_T f(t) e^{-i2\pi k \frac{t}{T}} dt$$

Fourier Transform (Analysis): Given a function $f \in L^1(\mathbb{R})$, the Fourier transform $\mathcal{F}$ is defined as:

$$F(\omega) = \mathcal{F}\{f(t)\} = \int_{\mathbb{R}} f(t) e^{-i2\pi k t} dt$$

Inverse Fourier Transform (Synthesis): Given a function ($F \in L^1(\mathbb{R})$), the inverse Fourier transform ($\mathcal{F}^{-1}$) is defined as:

$$\hat{f}(t) = \mathcal{F}^{-1}\{F(k)\} = \int_{\mathbb{R}} F(k) e^{i2\pi k t} dk$$

Any periodic function can be represented exactly using Fourier series.

General Properties of Fourier Transform

Spectrum: $S_f(k) = |F(k)|^2$, $F(k) = \mathcal{F}\{f(t)\}$

Bandwidth: $F(k) = 0 \quad \forall k \notin B$

Duality Property: The Fourier transforms are related by the following relations:

$$\mathcal{F}\{f\} = \mathcal{F}^{-1}\{f^{-}\}, \quad \text{where } f^{-}(x) = f(-x)$$

$$\mathcal{F}\{\mathcal{F}\{f\}\} = f^{-}$$

Parity of a Function:

$f(x)$ is *even* if $f(-x) = f(x)$.

$f(x)$ is *odd* if $f(-x) = -f(x)$.

Fourier Transform follows the following properties:

Theorem	$f(x)$	$F(k)$
Similarity	$f(ax)$	$\frac{1}{ a } F\left(\frac{k}{a}\right)$
Addition	$f(x) + g(x)$	$F(k) + G(k)$
Homogeneity	$af(x)$	$aF(k)$
Time/phase shift	$f(x - a)$	$e^{i2\pi ak} F(k)$
Convolution	$f(x) * g(x)$	$F(x) \cdot G(x)$
Derivative	$f'(x)$	$i2\pi k \cdot F(k)$

Rayleigh's Theorem (Energy):

$$\int_{-\infty}^{\infty} |f(x)|^2 dx = \int_{-\infty}^{\infty} |F(k)|^2 dk$$

Parseval's Theorem (Power):

$$\int_{-\infty}^{\infty} f(x) g^*(x) dx = \int_{-\infty}^{\infty} F(k) G^*(k) dk$$

2. Multidimensional Fourier Transform

For a signal $f(x)$ with $x = [x_1, x_2, \dots, x_N]^T \in \mathbb{R}^n$.

The Fourier transform is:

$$F(\omega) = \mathcal{F}\{f(x)\} = \int_{\mathbb{R}^n} f(x) e^{-i2\pi x^T \omega} dx$$

The inverse Fourier transform reconstructs the signal:

$$f(x) = \int_{\mathbb{R}^n} F(\omega) e^{i2\pi x^T \omega} d\omega$$

Note that the exponential function ($e^{i2\pi x^T \omega}$) is determined by the product of the vectors x and ω where $\omega = [\omega_1, \dots, \omega_n]^T$ is the multidimensional frequency vector: $x \cdot \omega = x_1 \omega_1 + \dots + x_n \omega_n$

Why have a vector of frequencies? For a fixed ω , the exponential function represents a family of parallel lines in the (x_1, x_2) -plane (or in the spatial domain), normal to ω , with spacing: distance = $\frac{1}{\sqrt{\omega_1^2 + \omega_2^2}} = \frac{1}{\|\omega\|}$

Periodicity: We may now parametrize x using the normalized frequency vector. Give the starting point $a = [a_1, a_2]^T$ we obtain that: $x(t) = a + t \frac{\omega}{\|\omega\|}$

Therefore, the complex exponential function adopts the following form: $e^{\pm i2\pi x^T(t)\omega} = e^{\pm i2\pi a^T \omega} \cdot e^{\pm i2\pi t \|\omega\|}$

The first term is *constant*, not depending on t . The second term is *periodic* with period $1/\|\omega\|$. Therefore, although the frequency now is a vector ω we can keep most of the terminology for the one-dimensional case.

Separability: If a multivariable function can be written as a product of single-value:

$$f(x_1, x_2, \dots, x_n) = f_1(x_1) f_2(x_2) \cdots f_n(x_n)$$

$$\mathcal{F}\{f(x_1, x_2, \dots, x_n)\} = \mathcal{F}\{f_1(x_1)\} \mathcal{F}\{f_2(x_2)\} \cdots \mathcal{F}\{f_n(x_n)\}$$

Theorem	$f(x)$	$F(\omega)$
Similarity	$f(ax)$ with $a \in \mathbb{R}^n$	$\frac{1}{ a_1 a_2 \cdots a_n } F\left(\frac{\omega_1}{a}, \frac{\omega_2}{a}, \dots, \frac{\omega_n}{a}\right)$
Addition	$f(x) + g(x)$	$F(\omega) + G(\omega)$
Shift	$f(x - a)$	$e^{i2\pi a^T \omega} F(\omega)$
Conv.	$f(x) * g(x)$	$F(\omega) G(\omega)$
Deriv.	$\frac{\partial}{\partial x_i} f(x)$	$i2\pi \omega_i \cdot F(\omega)$

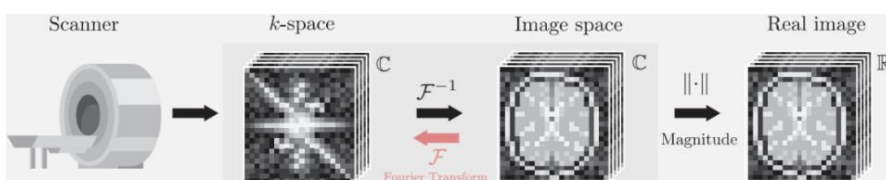
Rayleigh's Theorem (Energy):

$$\int_{\mathbb{R}^n} |f(x)|^2 dx = \int_{\mathbb{R}^n} |F(\omega)|^2 d\omega$$

Parseval's Theorem (Power):

$$\int_{\mathbb{R}^n} f(x) g^*(x) dx = \int F(\omega) G^*(\omega) d\omega$$

General Stretch Theorem: For any linear transformation, A , of the input data, the Fourier transform is: $\mathcal{F}\{f(Ax)\} = \frac{1}{|\det(A)|} F(A^{-T} \omega)$.



Special case – Rotations: The rotation matrix, R , has the property that $R = R^T$ and $\det(R) = 1$, i.e., A rotation matrix R in the spatial domain results in the same rotation in the frequency domain: $\mathcal{F}\{f(Rx)\} = F(R\omega)$.

5. Signal Spaces

A signal space is an inner product space where all elements have finite norm. Each element is called a signal.

Examples of signal spaces:

Finite length real-values discrete time signals:

$$\mathbf{x} = [x_1, x_2, \dots, x_N]^T, \mathbf{y} = [y_1, y_2, \dots, y_N]^T$$

$$\text{Inner Product Space: } \langle \mathbf{x}, \mathbf{y} \rangle = \sum_{n=1}^N x_n y_n = \mathbf{x}^T \mathbf{y} = \mathbf{y}^T \mathbf{x} \quad (\text{matrix form})$$

$$\text{Energy of } \mathbf{x}: \|\mathbf{x}\|^2 = \langle \mathbf{x}, \mathbf{x} \rangle = \sum_n x_n^2$$

If $\mathbf{x}$ and $\mathbf{y}$ are complex-valued:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{n=1}^N x_n y_n^* = \mathbf{x}^H \mathbf{y} \quad (\text{matrix form})$$

$$\|\mathbf{x}\|^2 = \sum_n |x_n|^2$$

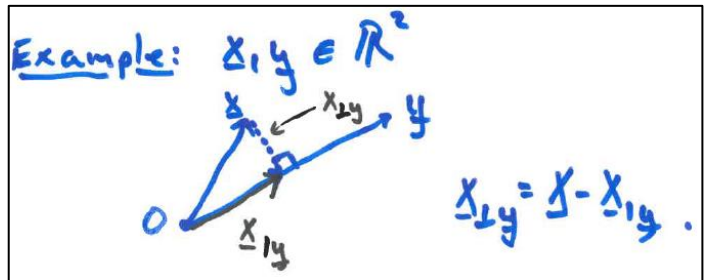
where $\mathbf{x}^H$ is the Hermitian transpose of $\mathbf{x}$ (i.e., transpose and complex conjugate *).

Inner Product Space: A signal space is often equipped with an inner product to measure similarity or orthogonality between signals.

Norm: The norm (magnitude) of a signal is given by $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$.

Orthogonality: Two signals $\mathbf{x}$ and $\mathbf{y}$ are orthogonal if: $\langle \mathbf{x}, \mathbf{y} \rangle = 0$. Orthogonal signals carry independent information, which is key in applications like multiplexing.

Projection of signal $\mathbf{x}$ onto signal $\mathbf{y}$: $\mathbf{x}_{\parallel \mathbf{y}} = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{y}\|^2} \mathbf{y}$,
if $\|\mathbf{y}\|^2 = 1$, then $\mathbf{x}_{\parallel \mathbf{y}} = \langle \mathbf{x}, \mathbf{y} \rangle \mathbf{y}$



Applications and Examples

1. Fourier Analysis:

- Sinusoidal basis functions $\phi_k(t) = e^{i2\pi kt}$ decompose signals into their frequency components.

2. Orthogonal Projections:

- Used in least squares for minimizing the error between a signal and its approximation.

3. Signal Approximation:

- Signals can be approximated by projecting onto a subspace spanned by orthogonal basis functions.

6. Detection theory

How do we make a decision?

- Daily life $\rightarrow$ no specific ruleset,
- Sometimes a particular rule or method

Engineers:

- Formulate decision rule
- Optimality criterion

The goal of detection theory is to select one of K *mutually exclusive hypotheses* in the set $\mathcal{H} = \{h_0, \dots, h_{K-1}\}$, based on some observations, $\mathbf{x} = [x_1, \dots, x_n]^T$. This data vector is deterministic but, we *model* it as a realization of a random vector, denoted with pdf $f_{\mathbf{X}}$.

Probability Model for Decision Problems: To do this, a decision rule is used, which is a function $\hat{H}(\mathbf{x})$, that assigns a hypothesis for each possible observation i.e.

$$\hat{H} : \text{range}(\mathbf{X}) \rightarrow \mathcal{H}.$$

We mark the output of a decision rule with a hat to indicate that the decision $\hat{H}(\mathbf{x})$ is not necessarily equal to the true hypothesis H . We may have some prior knowledge about the likeliness of a hypothesis being true: P_k is the **prior probability** of the hypothesis. The prior probabilities sum to one: $P_0 + \dots + P_{K-1} = 1$. After making the observations, our view of the probability of a given hypothesis being true changes. Now we say

$$\mathbb{P}(\hat{H} \text{ is true given } \mathbf{x}) = \mathbb{P}(H = h_k | \mathbf{x}),$$

Also known as the a posteriori probability of h_k being true. Where: $f_{\mathbf{X}|H}(\mathbf{x}|h_k)$ is the **likelihood** for h_k and is a conditional pdf of $\mathbf{X}$. $\mathbf{X}$ depends on which of the hypotheses are true, thus it's the conditional pdf of $\mathbf{X}$ under hypothesis h_k (likelihood for h_k), $f_{\mathbf{X}|H}(\mathbf{x}|h_k)$.

Bayes' theorem can expand the a posteriori probability and include the *prior* and *likelihood*:

$$\mathbb{P}(H = h_k | \mathbf{x}) = \frac{f_{\mathbf{X}|H}(\mathbf{x}|h_k)P_k}{f_{\mathbf{X}}(\mathbf{x})}$$

The denominator will be disregarded since it's the same regardless of h_k and will not enter the decision rules when derived. If we consider the decision rule evaluated at the random data, $\mathbf{X}$, then the decision $\hat{H}(\mathbf{X})$ is a random variable.

Maximum A Posteriori Probability (MAP) Decision Rule:

The MAP decision rule picks the hypothesis which has the highest probability of being correct. That is:

$$\hat{H}_{MAP}(\mathbf{x}) = \arg \max_{h_k \in \mathcal{H}} f_{\mathbf{X}|H}(\mathbf{x}|h_k)P_k$$

The prior probability affects the probability significantly. $P_k = 0$ means the hypothesis will never be

selected, and the opposite if $P_k = 1$. Furthermore, the more probable a hypothesis is (*before we observe the data*), the more convincing the data needs to be in order for us to change our minds.

Decision Regions: The regions of the $\mathbf{x}$ -axis for which the rule will select a respective hypothesis.

Maximum Likelihood (ML) Decision Rule: In cases where **no prior probability** is available or undesirable to use, one can instead set all prior probabilities equal. This results in the ML-decision rule:

$$\hat{H}_{ML}(\mathbf{x}) = \arg \max_{h_k \in \mathcal{H}} f_{\mathbf{X}|H}(\mathbf{x}|h_k)$$

Bayes' Decision Rule: Instead of maximizing a probability, **Bayes' decision rule minimizes a cost/risk of choosing a particular hypothesis**. For this, a cost C_{kk} is assigned for if h_k is decided and h_k is true. Diagonal cost should be low, since these correspond to "true" decisions.

	$H = h_0$	$H = h_1$	...
$\hat{H} = h_0$	C_{00}	C_{01}	
$\hat{H} = h_1$	C_{10}	C_{11}	
$\vdots$			

The expected cost/risk for decision h_k given the data is obtained as the sum of the costs weighted by the conditional probability for hypothesis H given $\mathbf{x}$:

$$C(h_k | \mathbf{x}) = \sum_{k'=0}^{K-1} C_{kk'} \mathbb{P}(H = h_{k'} | \mathbf{x}).$$

Bayes' decision rule then chooses the hypothesis with the lowest expected cost:

$$\hat{H}_{Bayes}(\mathbf{x}) = \arg \min_{h_k \in \mathcal{H}} C(h_k | \mathbf{x}).$$

Thresholds for the Binary Case

The binary decision rule can be written as:

$$\hat{H} = \begin{cases} h_0, & \text{if } \Lambda(\mathbf{x}) \geq \gamma \\ h_1, & \text{otherwise} \end{cases}, \quad \Lambda(\mathbf{x}) = \frac{f_{\mathbf{X}|H}(\mathbf{x}|H_0)}{f_{\mathbf{X}|H}(\mathbf{x}|H_1)}$$

where a *likelihood ratio* denoted by $\Lambda(\mathbf{x})$ is compared to a *threshold* γ which does not depend on $\mathbf{x}$.

The threshold γ is **1 for ML**, $\frac{P_1}{P_0}$ for MAP, and for

$$\text{Bayes: } \gamma_{\text{Bayes}} = \frac{(C_{11} - C_{01})P_1}{(C_{00} - C_{10})P_0}.$$

Other Decision Rules MAP, ML, and Bayes are not the only decision rules. Two prominent examples of other decision rules are: 1. The Neyman-Pearson rule, 2. The Mini-max rule

7. Minimum Mean Squared Error (MMSE) and Linear MMSE estimation

MMSE is optimal among all possible estimators for θ , in the sense that it minimizes the mean squared error of the parameter, it is an estimator that is difficult to apply. We need to compute a conditional expectation of the unknown parameter given the data, for which we need a full statistical description of the dependency. Joint pdf of the data and unknown parameter.

MMSE Estimator

Why estimation?

- Provide solutions that work in general (variety of environments), adaptive to different cases
- The estimation of unknown values found in the environment, is a crucial task

Examples:

- User position for GPS
- The direction of arrival for sound signals on an array of microphones
- The speed of a ship navigating through the ocean, controlled by automatic control system

Estimation Problem Formulation:

Given a set of data $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, estimate the value of an unknown quantity θ .

What is an estimator? (Definition)

An estimator is a function $\hat{\theta} = g(\mathbf{x})$ of data $\mathbf{x}$ used to 'guess' the value of some unknown entity.

Estimation error definitions

- Estimation error: $\hat{\theta} - \theta$ (one-shot)
- Bias (mean error): $\mathbb{E}[\hat{\theta} - \theta]$ (tendency)
- Mean Squared error: $\mathbb{E}[(\hat{\theta} - \theta)^2]$ (circumvents positive and negative estimation errors cancelling out)

MMSE

The MMSE of θ given the data $\mathbf{x}$ is the function $\hat{\theta} = g(\mathbf{x})$ such that the MSE $\mathbb{E}[(\hat{\theta} - \theta)^2]$ is minimum. From this we define the MMSE estimator as:

$$\hat{\theta}_{MMSE}(\mathbf{X}) = \arg \min_{\hat{\theta}} \mathbb{E}[(\hat{\theta} - \theta)^2]$$

Using the law of total expectation, we get:

$$\mathbb{E}[(\hat{\theta} - \theta)^2] = \mathbb{E}[\mathbb{E}[(\hat{\theta} - \theta)^2 | \mathbf{X}]]$$

From this we can find the minimum by differentiating and equating to 0.

$$\frac{\partial \mathbb{E}[\mathbb{E}[(\hat{\theta} - \theta)^2 | \mathbf{X}]]}{\partial \hat{\theta}} = 2\hat{\theta} - 2\mathbb{E}[\theta | \mathbf{X}] = 0$$

$$\hat{\theta}_{MMSE} = \mathbb{E}[\theta | \mathbf{X}]$$

Properties of MMSE:

- Unbiased: $\mathbb{E}[\theta - \hat{\theta}] = \mathbb{E}[\theta - \mathbb{E}[\theta | \mathbf{x}]] = \mathbb{E}[\theta] - \mathbb{E}[\theta] = 0$
- Lowest MSE among all estimators, by definition
- Fulfills the "orthogonality principle", the error is uncorrelated with any function of the data

Remarks to the MMSE:

- Simple to derive, but needs full statistical description: $\mathbb{E}[\theta | \mathbf{x}] = \int \theta \underbrace{p(\theta | \mathbf{X})}_{\text{posterior}} d\theta = g(\mathbf{x})$
- Analytical solutions are not always available
- $\mathbb{E}[\mathbf{x} | \theta]$, may be computed through Monte Carlo sims.

Conditional expectation: $\mathbb{E}[\theta | \mathbf{x}] = \int \theta \underbrace{p(\theta | \mathbf{X})}_{\text{posterior}} d\theta$

We can use Bayes' Rule:

$$p(\theta | \mathbf{X}) = \frac{p(\theta, \mathbf{x})}{p(\mathbf{x})} = \frac{p(\theta, \mathbf{x})}{\int p(\theta, \mathbf{x}) d\theta}$$

In order to compute the MMSE we need the posterior, and in order to obtain then we need the joint pdf of θ and $\mathbf{x}$ (full statistical description, no good).

Linear MMSE estimator

- Keeps some good aspect
- Much simpler

How do we simplify? Constrain the functional form that the estimators can take

Properties of LMMSE:

- Only first and second order statistical modeling is necessary (**Mean of theta, mean of data, covariance matrix of the data, and the covariance between the data and theta**)
- Computational complexity of the estimator is fixed and low, N product and additions (from vector multiplication), for every realization. Covariance does not change "on the fly"
- LMMSE estimator is unbiased
- Minimises all MSE estimators among all affine estimators.
- Fulfills orthogonality principle for affine functions of data $\mathbf{X}$

To solve the problems of MMSE, the estimation function can be constrained to a linear form:

$$\hat{\theta}(\mathbf{x}) = h_0 + \sum_{n=1}^N h_n x_n = h_0 + \mathbf{h}^T \mathbf{x}$$

This changes the problem to:

$$\hat{\theta}_{LMMSE}(\mathbf{X}) = \arg \min_{h_0, \mathbf{h}} E[(h_0 + \mathbf{h}^T \mathbf{x} - \theta)^2]$$

Solving this problem is rather simple given: $h_0 = E[\theta] - \mathbf{h}^T E[\mathbf{x}]$ and $\mathbf{h} = \mathbf{C}_{xx}^{-1} \mathbf{C}_{x\theta}$, ($\mathbf{C}_{x\theta}^T = \mathbf{C}_{\theta x}$):

$$\hat{\theta}_{LMMSE}(\mathbf{X}) = E[\theta] + \mathbf{C}_{\theta x} \mathbf{C}_{xx}^{-1} (\mathbf{x} - E[\mathbf{x}])$$

Where: $\mathbf{C}_{\theta x} = E[(\mathbf{x} - E[\mathbf{x}])(\theta - E[\theta])^T]$, (covariance vector)
 $\mathbf{C}_{xx} = E[(\mathbf{x} - E[\mathbf{x}])(\mathbf{x} - E[\mathbf{x}])^T]$, (covariance matrix)

This means that to compute we only need $E[\theta]$, $E[\mathbf{x}]$, $\mathbf{C}_{xx}$, $\mathbf{C}_{\theta x}$. The LMMSE is unbiased, and fulfils a version of the orthogonality principle, where we constrain the function $h(\mathbf{x})$ to be linear.

The MSE of the LMMSE can also be directly calculated as:

$$MSE_{LMMSE} = E[(\hat{\theta}(\mathbf{x}) - \theta)^2] = \text{var}(\theta) - \mathbf{C}_{\theta x} \mathbf{C}_{xx}^{-1} \mathbf{C}_{x\theta}$$

So the MSE of the LMMSE is available before implementation.

LMMSE for multiple variables

To extend for multiple variables, θ becomes $\boldsymbol{\theta} = [\theta_1, \dots, \theta_K]^T$, which means the error we want to minimise becomes:

$$MSE_{\hat{\boldsymbol{\theta}}} = \sum_{k=1}^K MSE_{\hat{\theta}_k} = \sum_{k=1}^K E[(\hat{\theta}_k(\mathbf{x}) - \theta_k)^2]$$

The estimated variables can here be calculated as:

$$\hat{\theta}_k = E[\theta_k] + \mathbf{C}_{\theta_k x} \mathbf{C}_{xx}^{-1} (\mathbf{x} - E[\mathbf{x}])$$

Each variable k can be but as a row in a matrix, leading to the matrix notation:

$$\hat{\boldsymbol{\theta}} = E[\boldsymbol{\theta}] + \mathbf{C}_{\boldsymbol{\theta} x} \mathbf{C}_{xx}^{-1} (\mathbf{x} - E[\mathbf{x}])$$

Where $\mathbf{C}_{\boldsymbol{\theta} x} = E[(\boldsymbol{\theta} - E[\boldsymbol{\theta}])(\mathbf{x} - E[\mathbf{x}])^T]$

The total MSE can be calculated directly as the sum of MSEs from the single case:

$$MSE_{\hat{\boldsymbol{\theta}}} = \sum_K MSE_{\hat{\theta}_k} = \text{Tr}\{\mathbf{C}_{\boldsymbol{\theta} \boldsymbol{\theta}} - \mathbf{C}_{\boldsymbol{\theta} x} \mathbf{C}_{xx}^{-1} \mathbf{C}_{x \boldsymbol{\theta}}\}$$

The trace is used as the error is on the diagonals of the matrices.

8. Kalman Filtering (scalar and vector versions)

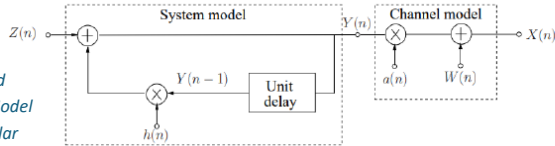
Why do we need a recursive LMMSE Estimator?

- Covariance matrix of data increases in size for every new data. And the number of constants increases.
- We do not like inverse covariance matrix, which would be necessary.
- Can we find a more efficient implementation of the estimates $\hat{y}(n)$ for all $n = 1, 2, \dots$?

Recursive LMMSE Estimator: $\hat{y}(n+1) =$ **linear function** $\{\hat{y}(n), x(n+1)\}$ (Kalman Filter).

Goal: To find LMMSE estimates of the samples of an **unobservable process** $Y(n)$ from the samples of a **related, observable process** $X(n)$, for $n = 1, 2, \dots$

Recursive implementation: at time $n+1$, the LMMSE estimate of $Y(n+1)$ is obtained as a linear function of the LMMSE estimate of $Y(n)$ and the new observation $X(n+1)$



State equation: $Y(n) = h(n)Y(n-1) + Z(n)$, $n = 1, 2, \dots$

Channel equation: $X(n) = a(n)Y(n) + W(n)$, $n = 1, 2, \dots$

Where:

- $h(n)$ and $a(n)$ are known, deterministic sequences
- $Z(n)$ and $W(n)$ are white noise processes with zero mean, and variances $\sigma_Z^2(n)$ and $\sigma_W^2(n)$
- $Y(0)$ is a r.v. with known mean $\mu_{Y(0)}$ and variance $\sigma_{Y(0)}^2$
- Assume $Y(n)$, $Z(n)$, and $W(n)$ mutually uncorrelated

System and Channel Model shows how:

- Each sample of $Y(n)$ is related to the previous sample
- How the observable $X(n)$ is related to $Y(n)$ through a weight and a summation with a noise process. Aka. Each of the samples of $X(n)$ is a weighted and noisy version of the unobservable $Y(n)$.

We want to find a **recursive implementation** of the LMMSE of $Y(n)$ based on observations $X(1), \dots, X(n)$. We define (where the hat denotes LMMSEE):

$\hat{Y}(n|n)$ **Estimation of $Y(n)$**

$\hat{Y}(n+1|n)$ **One-step prediction of $Y(n+1)$ at time n**

$\hat{X}(n+1|n)$ **One-step prediction of $X(n+1)$**

And the recursive implementation:

$$\hat{Y}(n+1|n+1) \equiv \mathcal{LF}\left\{ \underbrace{\hat{Y}(n|n)}_{\text{Estimation at time } n}, \underbrace{X(n+1)}_{\text{Observation at time } n+1} \right\}$$

Such a recursive implementation of the LMMSEE exists. It is called the **Kalman Filter**.

The recursion is a two-step process: One-step prediction and updating.

The MSE $\mathbb{E}[(Y(n) - \hat{Y}(n|n))^2] = R(n|n)$ can also be computed recursively.

Prediction step:

$$\hat{Y}(n+1|n) = h(n+1)\hat{Y}(n|n)$$

$$R(n+1|n) = h(n+1)^2 R(n|n) + \sigma_Z^2(n+1) \quad (\text{MSE})$$

$$\hat{X}(n+1|n) = a(n+1)\hat{Y}(n+1|n)$$

$\hat{Y}(n+1|n)$ satisfies the orthogonality principle and is therefore the LMMSEE.

Updating step:

$$b(n+1) \equiv \frac{a(n+1)R(n+1|n)}{a(n+1)^2 R(n+1|n) + \sigma_W^2(n+1)} \quad (\text{the Kalman gain})$$

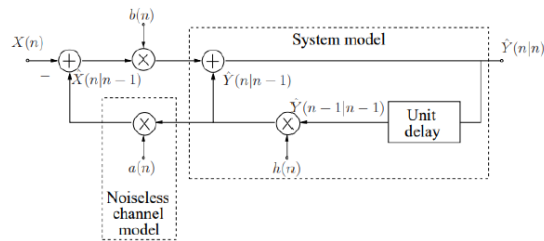
$\hat{Y}(n+1|n+1) = \hat{Y}(n+1|n) + b(n+1)[X(n+1) - \hat{X}(n+1|n)]$ LMMSEE of $Y(n+1)$, based on the prediction AND a correction factor. Satisfies Orthogonality Principle $\rightarrow$ LMMSEE.

$$R(n+1|n+1) = [1 - b(n+1)a(n+1)]R(n+1|n)$$

Initialization:

$$\hat{Y}(0|0) = \mu_{Y(0)}$$

$$R(0|0) = \sigma_{Y(0)}^2$$



← Block Diagram of the scalar Kalman Filter:

Vector version

When extending to a vector version, the models become:

$$y(n) = \mathbf{H}(n)\mathbf{y}(n-1) + \mathbf{z}(n), n = 1, 2, \dots$$

$$x(n) = \mathbf{A}(n)\mathbf{y}(n) + \mathbf{w}(n), n = 1, 2, \dots$$

$\mathbf{H}(n)$ and $\mathbf{A}(n)$ are now known matrices describing the relation between previous estimates and new. $\mathbf{z}(n)$ and $\mathbf{w}(n)$ is now vectors of noise. The steps of the Kalman filter changes to:

Prediction Step:

$$\hat{\mathbf{y}}(n|n-1) = \mathbf{H}(n)\hat{\mathbf{y}}(n-1|n-1)$$

$$\hat{\mathbf{x}}(n|n-1) = \mathbf{A}(n)\hat{\mathbf{y}}(n|n-1)$$

$$\mathbf{R}(n|n-1) = \mathbf{H}(n)\mathbf{R}(n-1|n-1)\mathbf{H}(n)^T + \mathbf{Q}_z(n)$$

Updating Step:

$$\hat{\mathbf{y}}(n|n) = \hat{\mathbf{y}}(n|n-1) + \mathbf{B}(n)(x(n) - \hat{\mathbf{x}}(n|n-1))$$

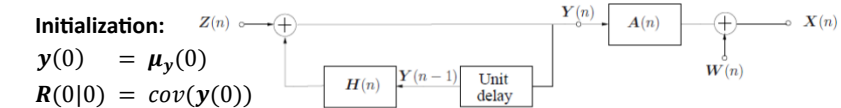
$$\mathbf{R}(n|n) = (\mathbf{I} - \mathbf{B}(n)\mathbf{A}(n))\mathbf{R}(n|n-1)$$

$$\mathbf{B}(n) = \mathbf{R}(n|n-1)\mathbf{A}(n)^T(\mathbf{A}(n)\mathbf{R}(n|n-1)\mathbf{A}(n)^T + \mathbf{Q}_w(n))^{-1}$$

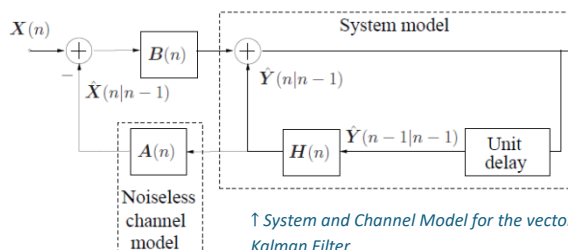
Initialization:

$$\mathbf{y}(0) = \boldsymbol{\mu}_y(0)$$

$$\mathbf{R}(0|0) = \text{cov}(\mathbf{y}(0))$$



↑ Block Diagram of the vector Kalman Filter



Application:
sensor fusion

9. One-step linear predictor

The **one-step linear predictor** is a method for predicting the next value in a time series based on the linear combination of previous values. It is commonly used in signal processing, such as speech analysis or AR (autoregressive) modeling.

Concept

The idea is to predict $x[n]$, the current sample, using a weighted sum of its p past samples:

$$\hat{x}[n] = - \sum_{k=1}^p a_k \cdot x[n-k]$$

$\hat{x}[n]$: the predicted value.

$x[n-k]$: the previous samples (past observations).

a_k : the **predictor coefficients**, which determine how much weight to assign to each previous sample.

The error between the actual and predicted value is the **prediction error**:

$$e[n] = x[n] - \hat{x}[n]$$

Steps to Derive the One-Step Linear Predictor

1. Model the Signal as an AR Process:

Assume the signal $x[n]$ is generated as:

$$x[n] = - \sum_{k=1}^p a_k \cdot x[n-k] + e[n]$$

a_k : predictor coefficients to be determined.

$e[n]$: prediction error (noise).

2. Formulate the Prediction:

Use the past p samples to predict the current sample:

$$\hat{x}[n] = - \sum_{k=1}^p a_k \cdot x[n-k]$$

The goal is to find a_k that minimizes the error $e[n] = x[n] - \hat{x}[n]$

3. Minimize the Mean Squared Error (MSE):

Apply the principle of least squares:

$$\text{Minimize } E[e[n]^2] =$$

$$E \left[\left(x[n] + \sum_{k=1}^p a_k \cdot x[n-k] \right)^2 \right]$$

This ensures the best-fit predictor coefficients.

4. Solve for Coefficients a_k :

- Use the **Yule-Walker equations** to relate a_k to the autocorrelation $r(m)$ of $x[n]$.
- Efficiently compute a_k using the **Levinson-Durbin algorithm**, avoiding matrix inversion.

5. Predict the Signal:

Substitute a_k and past samples into the predictor formula:

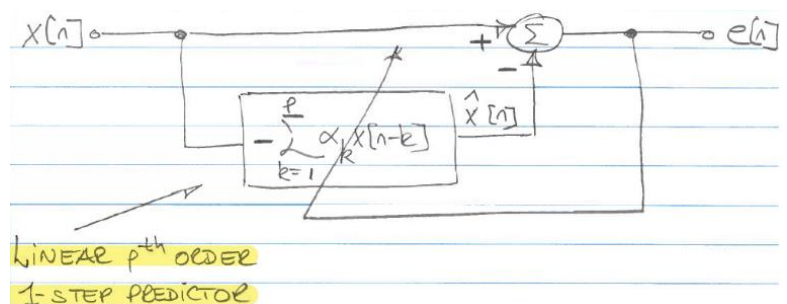
$$\hat{x}[n] = - \sum_{k=1}^p a_k \cdot x[n-k]$$

The result is the one-step prediction of $x[n]$.

Applications

- **Speech Processing:** Predictive coding for compression (e.g., LPC in audio codecs).
- **Error Prediction:** Useful in noise reduction and signal denoising.
- **AR Models:** Fundamental for spectral estimation and time-series analysis.

The **one-step linear predictor** is an essential tool in AR-based systems, offering both predictive capability and efficient error minimization.



10. The Levinson-Durbin algorithm

The Levinson-Durbin algorithm is an efficient method for solving the **Yule-Walker equations** used in linear prediction analysis, especially in autoregressive (AR) modeling. Instead of performing expensive matrix inversion, it recursively computes the predictor coefficients for increasing model orders.

Key Steps of the Levinson-Durbin Algorithm

1. Initialization:

- Start with the autocorrelation $r(m)$ of the signal.
- Set the initial prediction error: $E_0 = r(0)$

2. Iterative Computation for Each Order m (from 1 to p):

- Calculate Reflection Coefficient k_m :**
Use past coefficients and autocorrelation:

$$k_m = -\frac{r(m) + \sum_{k=1}^{m-1} a_{m-1,k} \cdot r(m-k)}{E_{m-1}}$$

k_m : a "dummy variable" that measures the contribution of the new coefficient.

- Update Predictor Coefficients:**

$$a_{m,m} = k_m$$

For $k = 1$ to $m - 1$:

$$a_{m,k} = a_{m-1,k} + k_m \cdot a_{m-1,m-k}$$

- Update the Prediction Error:**

$$E_m = E_{m-1} \cdot (1 - k_m^2)$$

3. Repeat Until Desired Order p is Reached:

Continue the recursion until all predictor coefficients $a_{p,k}$ are computed for the p -th order model.

Result

- The final coefficients $a_{p,k}$ for $k = 1, 2, \dots, p$ are the **Linear Prediction Coefficients (LPC)** for the p -th order AR model.
- All lower-order LPCs (for 1 to $p - 1$) are also computed during the process.

Advantages

- Efficient (computational cost is $O(p^2)$ compared to $O(p^3)$ for matrix inversion).
- Recursive computation avoids recalculating lower-order models from scratch.

The Levinson-Durbin algorithm is widely used in speech processing, signal prediction, and AR modeling due to its computational efficiency and numerical stability.

$E_0 = r(0)$	Initial prediction error
for $m = 1$ to p	
$k_m = -\frac{r(m) + \sum_{k=1}^{m-1} a_{m-1,k} \cdot r(m-k)}{E_{m-1}}$	Reflection Coefficient
$a_{m,m} = k_m$	Update Predictor Coefficients
for $k = 1$ to $m - 1$	
$a_{m,k} = a_{m-1,k} + k_m \cdot a_{m-1,m-k}$	Update Predictor Coefficients
end	
$E_m = E_{m-1} \cdot (1 - k_m^2)$	Update the Prediction Error
end	

11. AR-filter design by modelling

Autoregressive (AR) filter design involves modeling a signal as the output of a linear system driven by white noise. The AR filter parameters are derived from the signal's statistical properties, such as autocorrelation, and are commonly used in signal processing, speech processing, and spectral analysis.

Key Steps for AR-Filter Design by Modeling

1. Model Assumption:

Assume the signal $x[n]$ can be represented as an AR process:

$$x[n] = \sum_{k=1}^p a_k x[n-k] + e[n]$$

a_k : AR coefficients (filter parameters)

p : Order of the AR model

$e[n]$: White noise (residual/error)

2. Autocorrelation-Based Estimation:

Compute the autocorrelation sequence $r(m)$ of the signal, which captures its statistical dependence.

3. Solve Yule-Walker Equations:

Use the autocorrelation function to solve the Yule-Walker equations:

$$r(m) = - \sum_{k=1}^p a_k r(m-k), \quad m = 1, 2, \dots, p$$

Yule-Walker eq.: $\mathbf{Ra} = \mathbf{r}$
 $\mathbf{R}$: Toeplitz autocorrelation matrix.
 $\mathbf{a}$: AR coefficient vector.
 $\mathbf{r}$: Autocorrelation vector of lagged terms.

These equations establish a relationship between the AR coefficients and the autocorrelation values.

4. Levinson-Durbin Algorithm (Efficient Computation):

Use the Levinson-Durbin algorithm to recursively calculate the AR coefficients a_k without directly solving the equations.

5. Filter Construction:

Construct the AR filter using the derived coefficients a_k :

$$H(z) = \frac{1}{1 + \sum_{k=1}^p a_k z^{-k}}$$

$H(z)$: the transfer function of the AR filter.

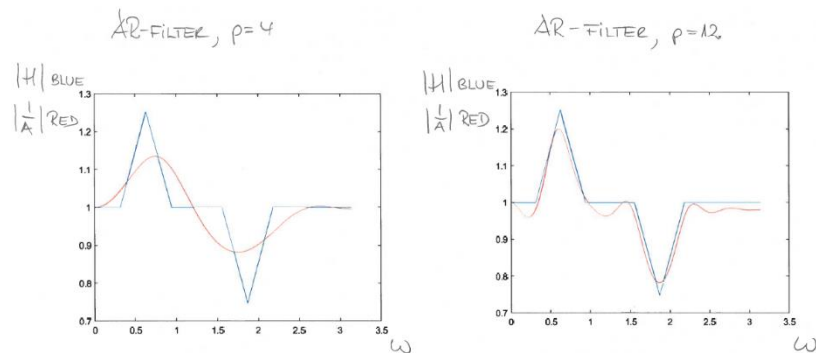
6. Stability Check:

Ensure the AR filter is stable by verifying that all poles lie within the unit circle in the z -plane.

7. Signal Modeling/Filtering:

Use the designed AR filter to:

- Generate synthetic signals by driving the filter with white noise.
- Analyze the signal's **power spectrum** (via the AR model).
- Predict or smooth the signal by applying the filter.



Applications

- **Speech Processing:** AR filters model vocal tract resonances.
- **Spectral Estimation:** AR modeling helps estimate the power spectrum of a signal.
- **Signal Prediction:** AR filters predict future values in time-series data.

By leveraging AR-filter design, signals can be effectively modeled, analyzed, and processed with high accuracy and computational efficiency.

AR-modelling

Given the relation between the autocorrelation and power spectrum: $R(n) \xleftrightarrow{DTFT} P(\omega)$, knowing the **desired amplitude spectrum** $|H(e^{j\omega})|$ we can derive the **power spectrum** $|H(e^{j\omega})|^2$ and then calculate the autocorrelation coefficients as:

$$R(n) = \text{IDTFT}(|H(e^{j\omega})|^2)$$

This autocorrelation function can then be used in the Yule-Walker equations to calculate the filter coefficients:

$$\mathbf{Ra} = -\mathbf{r} \Rightarrow \mathbf{A}(z)$$

Which then can be used to calculate the expected power spectrum:

$$\hat{P}(\omega) = \left| \frac{G}{A(e^{j\omega})} \right|^2 = \frac{G^2}{|1 + \sum_{k=1}^p a_k e^{-jk\omega}|^2}$$

NB: is worse at hitting dips than spikes (only consists of poles)

12. The Lattice FIR filter

Core Concept:

The Lattice FIR filter uses a recursive structure to compute forward and backward prediction errors at each stage m , minimizing the residual error.

Equations for Writing:

- **Forward Prediction Error:** $e^{(m)}[n] = e^{(m-1)}[n] - k_m \tilde{e}^{(m-1)}[n-1]$.
- **Backward Prediction Error:** $\tilde{e}^{(m)}[n] = \tilde{e}^{(m-1)}[n-1] - k_m e^{(m-1)}[n]$.
- Final forward error gives the filter output: $y[n] = e^{(m)}[n]$.

This set of differences equations expresses the m -th order forward and backward prediction errors can be drawn as a **Cascade of m Stages** (one iteration of the Levinson-Durbin algorithm)

- **Horizontal Arrows:** Represent the forward and backward error path.
- **Diagonal Arrows:** Scale the forward and backward paths by the Reflection Coefficient k_m .
- **Delay Block (z^{-1}):** Indicate one-sample delay for the backward error.
- At each stage, the forward error $e^{(m-1)}[n]$ is updated using the previous stage's forward and backward errors, scaled k_m (Reflection Coefficient).
- The cascade structure allows us to compute filter coefficients recursively, ensuring efficiency and stability.

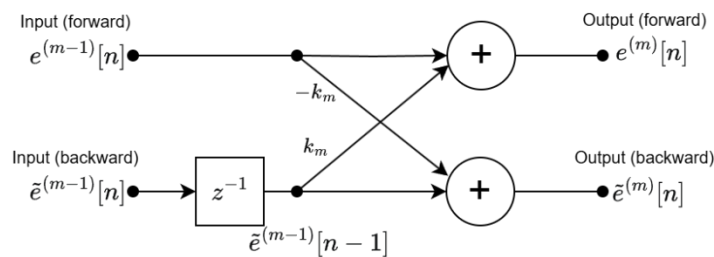
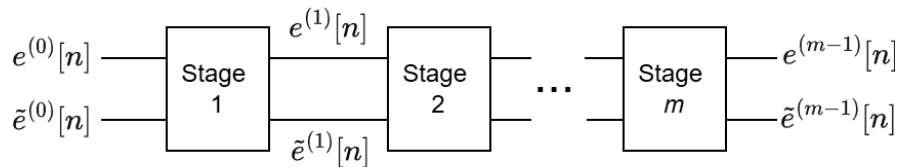


Figure 1: Cascade of m stages

In conclusion we can now state that

instead of $A^{(m)}(z)$ being an **ordinary tapped-delay-line FIR-filter** it can be implemented as a cascade of m sections of the structure figure 1 – This is known as the Lattice Filter. **Note: For $m = 0$,**

there's no prediction, so the error is directly the input signal: $e^{(0)}[n] = \tilde{e}^{(0)}[n] = x[n]$



FIR coefficients

The coefficients are calculated from the residual signals(error), and for order m becomes:

$$k_m = -\frac{\sum_{n=0}^{N-1} (e^{(m-1)}[n] \cdot \tilde{e}^{(m-1)}[n-1])}{\sum_{n=0}^{N-1} (\tilde{e}^{(m-1)}[n])^2}$$

NB: Should be numerical smaller than zero.

NOTE:

- i th filter coefficient is calculated based on residual signals from the $(i - 1)^{st}$ lattice stage!
- The forward and backward **Residual variances** are identical for order m .
- Compared to FIR implementation of LPC, we do not use the columns from the Levinson-Durbin algorithm, but the diagonal (k_m) ensuring values between -1 and 1
- This is good for fixed-point calculations (proof omitted)